

```
In [26]: import pandas as pd
import numpy as np

# Load your CSV file (replace 'your_data.csv' with your actual file name)
df = pd.read_csv('cus.csv')
df
```

Out[26]:

	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Location	Size	
0	1	55	Male	Blouse	Clothing	53	Kentucky	L	
1	2	19	Male	Sweater	Clothing	64	Maine	L	Ma
2	3	50	Male	Jeans	Clothing	73	Massachusetts	S	Ma
3	4	21	Male	Sandals	Footwear	90	Rhode Island	M	Ma
4	5	45	Male	Blouse	Clothing	b49	Oregon	M	Turq
...	...	...	...	...	...	...	...	...	
3895	3896	40	Female	Hoodie	Clothing	28	Virginia	L	Turq
3896	3897	52	Female	Backpack	Accessories	49	Iowa	L	V
3897	3898	46	Female	Belt	Accessories	33	New Jersey	L	C
3898	3899	44	Female	Shoes	Footwear	77	Minnesota	S	B
3899	3900	52	Female	Handbag	Accessories	81	California	M	I

3900 rows × 18 columns

```
In [27]: df.info() # 3863 shows some null valu
```

```
<class 'pandas.DataFrame'>
RangeIndex: 3900 entries, 0 to 3899
Data columns (total 18 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Customer ID                          3900 non-null   int64
1   Age                                   3900 non-null   int64
2   Gender                               3900 non-null   str
3   Item Purchased                       3900 non-null   str
4   Category                             3900 non-null   str
5   Purchase Amount (USD)                3900 non-null   str
6   Location                             3900 non-null   str
7   Size                                 3900 non-null   str
8   Color                                3900 non-null   str
9   Season                               3900 non-null   str
10  Review Rating                        3863 non-null   str
11  Subscription Status                  3900 non-null   str
12  Shipping Type                       3900 non-null   str
13  Discount Applied                    3900 non-null   str
14  Promo Code Used                     3900 non-null   str
15  Previous Purchases                   3900 non-null   str
16  Payment Method                      3900 non-null   str
17  Frequency of Purchases               3900 non-null   str
dtypes: int64(2), str(16)
memory usage: 861.4 KB
```

```
In [28]: # Check for missing values per column
print("\nMissing values per column:")
df.isnull().sum()
```

Missing values per column:

```
Out[28]: Customer ID          0
         Age                 0
         Gender              0
         Item Purchased      0
         Category            0
         Purchase Amount (USD) 0
         Location            0
         Size                0
         Color               0
         Season              0
         Review Rating       37
         Subscription Status  0
         Shipping Type       0
         Discount Applied    0
         Promo Code Used     0
         Previous Purchases  0
         Payment Method      0
         Frequency of Purchases 0
         dtype: int64
```

```
In [29]: # Check for duplicate rows
print(f"\nNumber of duplicate rows: {df.duplicated().sum()}")
```

Number of duplicate rows: 0

Validation check

Function to check_invalid_alphabet_columns

```
In [30]: def fun_check_invalid_alphabet_columns(df):
        for col in df.select_dtypes(include=["object", "string"]):
            mask = df[col].astype(str).str.contains(r'^A-Za-z\s', na=False) # t
            invalid = df[mask]
            if not invalid.empty:
                #unique_invalid = invalid[col].unique().tolist()
                unique_invalid = invalid[col].unique().tolist()[:30]
                print(f"--- Column: '{col}' | Unique Invalid Text: {unique_invalid}")
                display(invalid[[col]])
                print(f"-----")

In [31]: fun_check_invalid_alphabet_columns(df)
```

--- Column: 'Item Purchased' | Unique Invalid Text: ['T-shirt'] ---

Item Purchased	
34	T-shirt
36	T-shirt
52	T-shirt
88	T-shirt
109	T-shirt
...	...
3771	T-shirt
3854	T-shirt
3858	T-shirt
3864	T-shirt
3871	T-shirt

147 rows × 1 columns

--- Column: 'Purchase Amount (USD)' | Unique Invalid Text: ['53', '64', '73', '90', 'b49', '20', '8c5', '34', '97', '31', '68', '72', '51', '81', '36', '38', '48', '62', '37', '88', '22', '25', '56', '94', '79', '67', '91', '33', '69', '45'] ---

Purchase Amount (USD)	
0	53
1	64
2	73
3	90
4	b49
...	...
3895	28
3896	49
3897	33
3898	77
3899	81

3900 rows × 1 columns

--- Column: 'Review Rating' | Unique Invalid Text: ['3.1', '3.1a', '3\\.5', '2.7', '2.9', '3.2', '2.6', '4.8', '4.1', '4.9', '4.5', '4.7', '2.8', '4.6', '3.3', '4.4', '3.6', '5', '4', '4.2', '3.7', '3.9', '3', '3.8', '3.5', '3.4', '4.3', '2.5'] ---

Review Rating	
0	3.1
1	3.1a
2	3.1
3	3\\.5
4	2.7
...	...
3895	4.2
3896	4.5
3897	2.9
3898	3.8
3899	3.1

3863 rows × 1 columns

--- Column: 'Shipping Type' | Unique Invalid Text: ['Express 2', '2-Day Shipping'] -
--

Shipping Type	
8	Express 2
9	2-Day Shipping
17	2-Day Shipping
34	2-Day Shipping
35	2-Day Shipping
...	...
3876	2-Day Shipping
3878	2-Day Shipping
3881	2-Day Shipping
3894	2-Day Shipping
3895	2-Day Shipping

628 rows × 1 columns

--- Column: 'Previous Purchases' | Unique Invalid Text: ['14', '2', '23', '49', '3hvj1', '19', '8', '4', '26', '10', '37', '31', '34', '44', '36', '17', '46', '50', '22', '32', '40', '16', '13', '7', '41', '45', '38', '48', '18', '15'] ---

Previous Purchases	
0	14
1	2
2	23
3	49
4	3hvj1
...	...
3895	32
3896	41
3897	24
3898	24
3899	33

3900 rows × 1 columns

--- Column: 'Frequency of Purchases' | Unique Invalid Text: ['Bi-Weekly', 'Every 3 Months'] ---

Frequency of Purchases	
10	Bi-Weekly
16	Bi-Weekly
19	Bi-Weekly
20	Every 3 Months
27	Every 3 Months
...	...
3874	Bi-Weekly
3888	Bi-Weekly
3893	Bi-Weekly
3894	Every 3 Months
3896	Bi-Weekly

1131 rows × 1 columns

In [32]: `df.columns`

Out[32]: Index(['Customer ID', 'Age', 'Gender', 'Item Purchased', 'Category',
'Purchase Amount (USD)', 'Location', 'Size', 'Color', 'Season',
'Review Rating', 'Subscription Status', 'Shipping Type',
'Discount Applied', 'Promo Code Used', 'Previous Purchases',
'Payment Method', 'Frequency of Purchases'],
dtype='str')

```
In [33]: import pandas as pd
import numpy as np

def fun_remove_invalid_character(df, *col_names):
    """
    Removes alphabets, /, \, and - from multiple specified columns.
    """
    for col_name in col_names:
        if col_name in df.columns:
            # Remove letters, forward slashes, backslashes, and hyphens
            df[col_name] = df[col_name].astype(str).str.replace(r'[A-Za-z/\\-]', '')

            # Convert empty strings or spaces left behind back to NaN
            df[col_name] = df[col_name].replace(r'^\s*$', np.nan, regex=True)

            print(f"Cleaned column: '{col_name}'")
        else:
            print(f"Warning: Column '{col_name}' not found in DataFrame.")

    return df
```

Function to remove_invalid_character in NUMBERS

```
In [35]: df = fun_remove_invalid_character(df, 'Purchase Amount (USD)', 'Review Rating', 'Pr
Cleaned column: 'Purchase Amount (USD)'
Cleaned column: 'Review Rating'
Cleaned column: 'Previous Purchases'
```

```
In [36]: fun_check_invalid_alphabet_columns(df) # to check again wether cleane
--- Column: 'Item Purchased' | Unique Invalid Text: ['T-shirt'] ---
```

Item Purchased	
34	T-shirt
36	T-shirt
52	T-shirt
88	T-shirt
109	T-shirt
...	...
3771	T-shirt
3854	T-shirt
3858	T-shirt
3864	T-shirt
3871	T-shirt

147 rows × 1 columns

```
-----
--- Column: 'Purchase Amount (USD)' | Unique Invalid Text: ['53', '64', '73', '90',
'49', '20', '85', '34', '97', '31', '68', '72', '51', '81', '36', '38', '48', '62',
'37', '88', '22', '25', '56', '94', '79', '67', '91', '33', '69', '45'] ---
```

Purchase Amount (USD)	
0	53
1	64
2	73
3	90
4	49
...	...
3895	28
3896	49
3897	33
3898	77
3899	81

3900 rows × 1 columns

--- Column: 'Review Rating' | Unique Invalid Text: ['3.1', '3.5', '2.7', '2.9', '3.2', '2.6', '4.8', '4.1', '4.9', '4.5', '4.7', '2.8', '4.6', '3.3', '4.4', '3.6', '5', '4', '4.2', '3.7', '3.9', '3', '3.8', '3.4', '4.3', '2.5'] ---

Review Rating	
0	3.1
1	3.1
2	3.1
3	3.5
4	2.7
...	...
3895	4.2
3896	4.5
3897	2.9
3898	3.8
3899	3.1

3863 rows × 1 columns

--- Column: 'Shipping Type' | Unique Invalid Text: ['Express 2', '2-Day Shipping'] --

Shipping Type	
8	Express 2
9	2-Day Shipping
17	2-Day Shipping
34	2-Day Shipping
35	2-Day Shipping
...	...
3876	2-Day Shipping
3878	2-Day Shipping
3881	2-Day Shipping
3894	2-Day Shipping
3895	2-Day Shipping

628 rows × 1 columns

--- Column: 'Previous Purchases' | Unique Invalid Text: ['14', '2', '23', '49', '31', '19', '8', '4', '26', '10', '37', '34', '44', '36', '17', '46', '50', '22', '32', '40', '16', '13', '7', '41', '45', '38', '48', '18', '15', '25'] ---

Previous Purchases	
0	14
1	2
2	23
3	49
4	31
...	...
3895	32
3896	41
3897	24
3898	24
3899	33

3900 rows × 1 columns

--- Column: 'Frequency of Purchases' | Unique Invalid Text: ['Bi-Weekly', 'Every 3 Months'] ---

Frequency of Purchases	
10	Bi-Weekly
16	Bi-Weekly
19	Bi-Weekly
20	Every 3 Months
27	Every 3 Months
...	...
3874	Bi-Weekly
3888	Bi-Weekly
3893	Bi-Weekly
3894	Every 3 Months
3896	Bi-Weekly

1131 rows × 1 columns

In [37]: `#### now convert column to float or int, if all values of the column are numeric`

In [38]: `for col in df.columns:
1. Convert to string, strip hidden spaces, and try converting to numeric
converted = pd.to_numeric(df[col].astype(str).str.strip(), errors='coerce')

2. If it actually found numbers (and isn't just a pure text column like names
if converted.notna().sum() > 0 and df[col].astype(str).str.strip().eq('').sum()
Check if it has decimals to decide int vs float
df[col] = converted

Check results
print(df.info())`

```
<class 'pandas.DataFrame'>
RangeIndex: 3900 entries, 0 to 3899
Data columns (total 18 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Customer ID                          3900 non-null   int64
1   Age                                  3900 non-null   int64
2   Gender                              3900 non-null   str
3   Item Purchased                       3900 non-null   str
4   Category                             3900 non-null   str
5   Purchase Amount (USD)                3900 non-null   int64
6   Location                             3900 non-null   str
7   Size                                 3900 non-null   str
8   Color                                3900 non-null   str
9   Season                               3900 non-null   str
10  Review Rating                        3863 non-null   float64
11  Subscription Status                  3900 non-null   str
12  Shipping Type                       3900 non-null   str
13  Discount Applied                    3900 non-null   str
14  Promo Code Used                     3900 non-null   str
15  Previous Purchases                   3900 non-null   int64
16  Payment Method                      3900 non-null   str
17  Frequency of Purchases               3900 non-null   str
dtypes: float64(1), int64(4), str(13)
memory usage: 835.9 KB
None
```

```
In [39]: fun_check_invalid_alphabet_columns(df)
```

--- Column: 'Item Purchased' | Unique Invalid Text: ['T-shirt'] ---

Item Purchased	
34	T-shirt
36	T-shirt
52	T-shirt
88	T-shirt
109	T-shirt
...	...
3771	T-shirt
3854	T-shirt
3858	T-shirt
3864	T-shirt
3871	T-shirt

147 rows × 1 columns

--- Column: 'Shipping Type' | Unique Invalid Text: ['Express 2', '2-Day Shipping'] -
--

Shipping Type	
8	Express 2
9	2-Day Shipping
17	2-Day Shipping
34	2-Day Shipping
35	2-Day Shipping
...	...
3876	2-Day Shipping
3878	2-Day Shipping
3881	2-Day Shipping
3894	2-Day Shipping
3895	2-Day Shipping

628 rows × 1 columns

--- Column: 'Frequency of Purchases' | Unique Invalid Text: ['Bi-Weekly', 'Every 3 M
onths'] ---

Frequency of Purchases	
10	Bi-Weekly
16	Bi-Weekly
19	Bi-Weekly
20	Every 3 Months
27	Every 3 Months
...	...
3874	Bi-Weekly
3888	Bi-Weekly
3893	Bi-Weekly
3894	Every 3 Months
3896	Bi-Weekly

1131 rows × 1 columns

Replace 'Express 2' with 'Express' in column 'Shipping Type' only

```
In [40]: # 1. First, strip any hidden spaces in the column
df['Shipping Type'] = df['Shipping Type'].astype(str).str.strip()

# 2. Then replace 'Express 2' with 'Express'
df['Shipping Type'] = df['Shipping Type'].str.replace('Express 2', 'Express', regex
```

```
In [41]: fun_check_invalid_alphabet_columns(df)
```

--- Column: 'Item Purchased' | Unique Invalid Text: ['T-shirt'] ---

Item Purchased	
34	T-shirt
36	T-shirt
52	T-shirt
88	T-shirt
109	T-shirt
...	...
3771	T-shirt
3854	T-shirt
3858	T-shirt
3864	T-shirt
3871	T-shirt

147 rows × 1 columns

--- Column: 'Shipping Type' | Unique Invalid Text: ['2-Day Shipping'] ---

Shipping Type	
9	2-Day Shipping
17	2-Day Shipping
34	2-Day Shipping
35	2-Day Shipping
38	2-Day Shipping
...	...
3876	2-Day Shipping
3878	2-Day Shipping
3881	2-Day Shipping
3894	2-Day Shipping
3895	2-Day Shipping

627 rows × 1 columns

--- Column: 'Frequency of Purchases' | Unique Invalid Text: ['Bi-Weekly', 'Every 3 M
onths'] ---

Frequency of Purchases	
10	Bi-Weekly
16	Bi-Weekly
19	Bi-Weekly
20	Every 3 Months
27	Every 3 Months
...	...
3874	Bi-Weekly
3888	Bi-Weekly
3893	Bi-Weekly
3894	Every 3 Months
3896	Bi-Weekly

1131 rows × 1 columns

```
In [42]: # Save DataFrame to a CSV file without the index column
df.to_csv('updated_data.csv', index=False)
```

```
print("DataFrame successfully saved as 'updated_data.csv'!")
```

DataFrame successfully saved as 'updated_data.csv'!